

AXIOM
SPACE

INTELLIGENT LEARNING AGENT SYSTEM

05

DOC 05

2026 年软件杯 · A3 组

部署说明书

Deployment Guide

说明安装、配置、启动、健康检查、故障恢复与演示环境复现方法。

项目：AXIOM Space

参赛队伍：给春稗以秋实

文档编号：05

版本：提交版 · 2026.07

DOCUMENT 05 / REVIEWER GUIDE

文档导读

本册职责 说明安装、配置、启动、健康检查、故障恢复与演示环境复现方法。

文档信息

文档编号：05
文档性质：安装、配置、启动与运行保障
核心职责：回答“一个没有参与开发的人，怎样在常规环境中把 AXIOM Space 可靠地运行起来？”
技术依据：[03-系统设计与开发说明书](#)
验证依据：[04-测试说明书与测试报告](#)

目录

Word 中可点击条目直接跳转；目录层级与正文书签保持一致。

- 01 部署基线与成功判定
- 02 1. 部署范围
- 03 2. 环境要求
- 04 3. 配置文件
- 05 4. 完整开发环境部署
- 06 5. 仅 Web 模式
- 07 6. 生产构建与演示运行
- 08 7. 数据初始化
- 09 8. 健康检查与成功标准
- 10 9. 服务与数据职责
- 11 10. 常见故障
- 12 11. 安全与提交前检查
- 13 12. 停止、恢复与复验

14 资料真源

部署基线与成功判定

AXIOM Space 当前交付形态是 Web 应用。完整模式由以下服务共同组成：

结构示意图

```
Next.js Web / Hono API
+ PostgreSQL (业务事实)
+ Redis / Worker (后台任务)
+ Qdrant (快速语义索引)
+ Ollama bge-m3 (本地嵌入)
+ LightRAG (图谱增强派生索引)
+ 外部大模型服务 (Agent 与资源生成)
```

部署与评审复现重点确认以下三项：

- 1. `pnpm dev` 是完整开发启动入口，会检查 Docker、启动基础设施、准备嵌入模型、同步数据库并同时启动 Worker 与 Web。
- 2. `pnpm dev:web` 只启动 Next.js，不会替你启动数据库、Worker、RAG 或生成 Prisma Client。
- 3. 部署成功不以“首页打开”为唯一标准，必须同时检查 Web、API、Agent、Worker、Qdrant、Ollama 和 LightRAG。

开发环境标准启动路径

代码示例 · POWERSHELL

```
Copy-Item -LiteralPath '.env.example' -Destination '.env'
pnpm install
pnpm dev
```

打开终端输出中的地址，通常为：

结构示意图

```
http://localhost:3000
```

如果 3000 端口被占用，完整启动脚本会从 3000—3019 中选择可用端口，应以实际终端输出为准。

1. 部署范围

1.1 完整模式

完整模式用于开发、测试和正式演示，包含：

- Web 页面与 Hono API；
- PostgreSQL 数据库；
- Redis 与后台任务 Worker；
- Qdrant 语义索引；

- Ollama 与 bge-m3 嵌入模型；
- LightRAG 图谱增强服务；
- 可配置的大模型服务；
- 资源生成和文件渲染依赖。

1.2 最小模式

最小模式只启动 Next.js：

代码示例 · POWERSHELL

```
pnpm dev:web
```

最小模式只适合查看不依赖后端的页面或在外部服务已经独立运行时使用。它不代表完整系统可用。

1.3 当前不包含的部署形态

- 移动端 App；
- 桌面端 / Electron；
- 小程序；
- 浏览器插件；
- 多租户学校集群。

2. 环境要求

2.1 必要软件

软件	用途	当前项目依据
Node.js	Next.js、脚本、Worker 和测试运行时	项目使用 Next.js 14、TypeScript 5；讯飞 WebSocket 适配器要求 Node 20+ 或可用 WebSocket polyfill
pnpm 9.11.0	依赖安装与脚本入口	package.json#packageManager
Docker Desktop / Docker Engine	PostgreSQL、Redis、Qdrant、Ollama、LightRAG	docker-compose.yml、scripts/dev-all.mjs
Git	获取代码和版本追溯	项目交付方式
Chromium / Chrome	Playwright E2E、Puppeteer PDF 等文件渲染	测试与资源渲染器

Windows 使用完整模式时，应先启动 Docker Desktop；项目脚本的错误提示建议启用 WSL 集成。

2.2 端口

服务	默认端口	健康检查
Web	3000	/、/api/health、/api/agent/health
PostgreSQL	5433	Prisma 数据库连接
Redis	6379	redis-cli ping 或容器健康状态
Qdrant	6333	/readyz
Ollama	11434	/api/tags
LightRAG	9621	/health

端口需要可用。Web 完整启动脚本会自动寻找 3000—3019 的可用端口，其他服务端口由环境变量和 Docker Compose 映射决定。

3. 配置文件

3.1 创建本地配置

项目脚本实际读取根目录 `.env`。首次部署可以执行：

代码示例 • POWERSHELL

```
Copy-Item -LiteralPath '.env.example' -Destination '.env'
```

`predev`、`build` 和 `postinstall` 使用的 `bootstrap` 脚本也会在 `.env` 缺失且 `.env.example` 存在时自动创建 `.env`。

不要把真实 `.env` 提交到版本库，也不要将密钥复制进截图、测试产物或文档。

3.2 数据库与应用地址

代码示例 • DOTENV

```
DATABASE_URL="postgresql://axiom:axiom_dev_password@localhost:5433/axiom?schema=public"
NEXT_PUBLIC_APP_URL="http://localhost:3000"
BETTER_AUTH_URL="http://localhost:3000"
NEXT_PUBLIC_BETTER_AUTH_URL="http://localhost:3000"
BETTER_AUTH_SECRET="替换为随机密钥"
```

生产环境的 Better Auth 地址必须与实际部署地址一致。`BETTER_AUTH_SECRET` 必须更换，不能沿用模板值。

3.3 主 AI 模型

代码示例 • DOTENV

```
AI_PROVIDER="openai"
AI_MODEL="填写实际模型标识"
AI_API_KEY="填写实际密钥"
```

`AI_PROVIDER`、`AI_MODEL` 和 `AI_API_KEY` 是新代码的主配置。遗留的 `VITE_AI_*` 名称仅作为兼容回退，不应作为新部署真源。

项目可以使用 OpenAI 兼容接口。实际 `provider`、`model` 和 `base URL` 应以最终提交环境为准，并在 09-AI 工具使用与 AI 辅助开发流程说明 中如实披露。

3.4 讯飞星火接入配置

项目已完成讯飞星火 WebSocket 适配接入。部署时通过以下环境变量启用：

代码示例 • DOTENV

```
XUNFEI_APP_ID=""
XUNFEI_API_KEY=""
XUNFEI_API_SECRET=""
XUNFEI_SPARK_VERSION="v3.5"
```

四项变量由部署环境显式注入；配置检测会在凭据完整时启用讯飞能力，在凭据不完整时保持主运行链路不受影响。

3.5 Redis、Qdrant、Ollama 与 LightRAG

模板中的关键配置为：

代码示例 • DOTENV

```
REDIS_URL="redis://localhost:6379"
QDRANT_BASE_URL="http://localhost:6333"
QDRANT_COLLECTION="axiom_cards"
OLLAMA_EMBEDDING_MODEL="bge-m3:latest"
LIGHTRAG_BASE_URL="http://localhost:9621"
LIGHTRAG_WORKSPACE_PREFIX="axiom"
LIGHTRAG_CHAT_CONTEXT="true"
```

LightRAG 有独立的 LLM 与 embedding 配置。它不是 AXIOM 的主数据库，只保存派生索引；关闭 LightRAG 不应删除数据库中的卡片事实。

3.6 开发模式

代码示例 • DOTENV

```
DEV_MODE="false"
```

只有本地受控演示环境可以临时使用 `DEV_MODE=true`。该模式可能在无会话 Cookie 时自动选择第一名用户，生产环境禁止开启。

4. 完整开发环境部署

4.1 安装依赖

在项目根目录执行：

代码示例 • POWERSHELL

```
pnpm install
```

安装后会运行 `postinstall`：准备 `.env` 并生成 Prisma Client。

4.2 启动 Docker Desktop

确认 Docker 服务端可用：

代码示例 · POWERSHELL

```
docker version
docker compose version
```

4.3 启动完整系统

代码示例 · POWERSHELL

```
pnpm dev
```

该命令按当前脚本执行以下动作：

4. `predev` 检查或创建 `.env`，生成 Prisma Client；
5. 检查 Docker 与 Docker Compose；
6. 启动 PostgreSQL、Redis、Qdrant、Ollama；
7. 检查 Ollama 是否已有 `bge-m3`，缺少时自动拉取；
8. 启动 LightRAG；
9. 执行 `prisma db push`；
10. 在 3000—3019 中寻找 Web 可用端口；
11. 启动后台任务 Worker；
12. 启动 Next.js 开发服务器。

首次拉取 Ollama 模型和 Docker 镜像可能耗时，应等待脚本完成，不要把下载时间误判为应用卡死。

4.4 单独启动基础设施

项目提供：

代码示例 · POWERSHELL

```
pnpm dev:infra
```

该脚本启动 PostgreSQL、Redis、Ollama 和 LightRAG，但当前脚本没有包含 Qdrant。需要 Qdrant 时应明确执行：

代码示例 · POWERSHELL

```
docker compose up -d qdrant
```

完整的 `pnpm dev` 会包含 Qdrant。

5. 仅 Web 模式

如果 Docker 服务和数据库已由其他方式提供，可以只启动 Web：

代码示例 • POWERSHELL

```
pnpm exec prisma generate
pnpm dev:web
```

注意：

- `pnpm dev:web` 不会运行 `predev`；
- 不会自动启动 Worker；
- 不会执行数据库同步；
- 不会启动 Qdrant、Ollama 和 LightRAG；
- 涉及认证、数据库、资源生成、RAG 和后台进度的功能可能不可用。

6. 生产构建与演示运行

6.1 数据库迁移

开发环境可使用：

代码示例 • POWERSHELL

```
pnpm db:push
```

生产环境应使用已经提交的迁移：

代码示例 • POWERSHELL

```
pnpm db:deploy
```

不要在不清楚数据影响时对正式数据库运行开发迁移命令。

6.2 生产构建

代码示例 • POWERSHELL

```
pnpm build
```

该命令会：

13. 准备环境文件；
14. 生成 Prisma Client；
15. 执行 Next.js 生产构建和类型检查。

构建成功后应存在 `.next/BUILD_ID`。

6.3 演示服务

代码示例 • POWERSHELL

```
pnpm demo:serve
```

该脚本要求已有生产构建，并会启动 PostgreSQL、Redis、Qdrant、Ollama、LightRAG、后台 Worker 和 `next start`。

在另一终端执行：

代码示例 • POWERSHELL

```
$env:DEMO_PORT = '3000'  
pnpm demo:preflight
```

如实际端口不同，应同步修改 `DEMO_PORT` 或 `DEMO_BASE_URL`。

7. 数据初始化

7.1 基础数据

代码示例 • POWERSHELL

```
pnpm db:seed
```

7.2 可用演示数据

代码示例 • POWERSHELL

```
pnpm db:seed:cs408  
pnpm db:verify:cs408  
pnpm db:seed:a3-golden  
pnpm db:seed:a3-inventory
```

不同种子承担不同演示目的。执行前应确认目标数据库，避免在正式数据上重复重置。

7.3 软件设计模式课程资料

`07-《软件设计模式》系统化学习资料` 是课程内容真源。它可以通过产品的资料导入入口进入当前 Vault。导入后应验证：

- 课程章节和模式层级被保留；
- `literature` 内容与用户自己的 `fleeting` / `permanent` 内容区分；
- 卡片、关系、星团和路径可以继续使用；
- 导入失败时存在明确结果和错误信息。

8. 健康检查与成功标准

8.1 自动检查

`pnpm demo:preflight` 会检查:

- AXIOM 页面;
- `/api/health`;
- `/api/agent/health`;
- Qdrant `/readyz`;
- Ollama `/api/tags`;
- LightRAG `/health`;
- 后台任务 Worker 进程;
- 关键页面与 API 预热后的二次检查。

8.2 Docker 服务状态

代码示例 • POWERSHELL

```
docker compose ps
```

PostgreSQL、Redis、Ollama 和其他服务应处于 `running` 或 `healthy` 状态。

8.3 部署成功定义

- ☐ Web 首页可访问。
- ☐ `/api/health` 返回成功。
- ☐ `/api/agent/health` 返回成功。
- ☐ 登录或受控演示账号可以进入 Vault。
- ☐ 数据库能读写当前 Vault 数据。
- ☐ Worker 正在运行。
- ☐ Qdrant、Ollama、LightRAG 健康检查通过，或系统明确显示其禁用状态。
- ☐ Agent 能开始流式回复。
- ☐ 资源生成能显示真实进度。
- ☐ 刷新后学习对象和任务状态仍存在。

9. 服务与数据职责

服务	保存什么	不能做什么
PostgreSQL	用户、Vault、卡片、路径、会话、画像、评估、资源任务等事实	不依赖向量库替自己保存事实
Redis / Worker	队列和后台任务执行	不成为长期业务事实源
Qdrant	快速语义向量索引	不反向覆盖卡片内容
Ollama	本地 embedding	不保存用户学习事实
LightRAG	可重建图谱与检索增强索引	不成为主数据库
文件系统	资源文件或本地 Vault 适配	不保存密钥到公开目录
外部模型	生成、分析、评估候选结果	不绕过业务工具直接写数据库

10. 常见故障

10.1 pnpm dev 提示 Docker 不可用

现象：脚本提示 Docker is required。**处理：**启动 Docker Desktop，确认 `docker version` 和 `docker compose version` 成功；Windows 检查 WSL 集成。

10.2 Prisma 无法连接 localhost:5433

现象：`Can't reach database server at localhost:5433`。**处理：**

代码示例 · POWERSHELL

```
docker compose up -d postgres
docker compose ps postgres
pnpm db:push
```

同时确认 `.env` 中 `DATABASE_URL` 端口与 `POSTGRES_PORT` 一致。

10.3 Prisma query engine DLL 出现 EPERM

现象：Windows 无法把 `query_engine-windows.dll.node.tmp...` 重命名为目标文件。**原因：**常见于正在运行的 Node / Next / Prisma 进程占用文件，或安全软件拦截。**处理：**停止占用项目 Prisma Client 的进程，关闭开发服务器后重新执行 `pnpm exec prisma generate`。不要直接删除整个用户目录或使用破坏性 Git 命令。

10.4 Puppeteer 找不到 Chrome

现象：HyperFrames PDF / 文件渲染测试提示 `Could not find Chrome`。**处理：**

代码示例 · POWERSHELL

```
pnpm exec puppeteer browsers install chrome
```

安装后重新执行渲染测试与 `pnpm test:acceptance:deep`。

10.5 LightRAG 不可用

检查:

代码示例 · POWERSHELL

```
docker compose ps lightrag ollama
Invoke-WebRequest -Uri 'http://localhost:9621/health'
Invoke-WebRequest -Uri 'http://localhost:11434/api/tags'
```

确认 LightRAG 的 LLM 密钥和 Ollama embedding 配置有效。LightRAG 失败不应删除数据库事实，但检索增强、隐藏关系和部分推荐会受影响。

10.6 Qdrant 未启动

`pnpm dev:infra` 当前不包含 Qdrant。执行:

代码示例 · POWERSHELL

```
docker compose up -d qdrant
Invoke-WebRequest -Uri 'http://localhost:6333/readyz'
```

10.7 AI 对话没有响应

检查:

- `AI_PROVIDER`、`AI_MODEL`、`AI_API_KEY`;
- 模型接口网络和配额;
- `/api/agent/health`;
- 服务端日志中的明确错误;
- 讯飞模式下四个 `XUNFEI_*` 变量和 `WebSocket` 运行时。

不要在日志或截图中展示完整密钥。

10.8 资源一直等待

检查 Worker、Redis、资源任务表和事件连接。刷新后任务状态应从数据库事实恢复; 如果只依赖临时 SSE 状态, 属于部署或实现错误。

11. 安全与提交前检查

- 生产环境更换数据库密码和 Better Auth Secret。
- 不提交 `.env`、API Key、访问令牌或带密钥的测试 artifact。
- `DEV_MODE` 在正式环境必须为 `false`。

- 对外只开放必要端口；数据库、Redis、Ollama 和内部 Sidecar 不应无保护暴露到公网。
- 日志、Agent 审计和错误信息需要脱敏。
- 备份数据库和用户资源后再执行迁移或批量种子脚本。
- 提交前执行测试文档中的完整复验门禁。

12. 停止、恢复与复验

12.1 停止开发进程

在运行 `pnpm dev` 的终端使用 `Ctrl+C`，脚本会向 Worker 和 Next.js 子进程发送终止信号。

12.2 停止 Docker 服务

代码示例 • POWERSHELL

```
docker compose down
```

该命令默认保留命名卷中的数据库和索引数据。不要在不需要清空数据时添加删除 `volumes` 的参数。

12.3 恢复

重新启动完整系统：

代码示例 • POWERSHELL

```
pnpm dev
```

系统应从 PostgreSQL 恢复业务事实，并从任务表、索引状态和文件清单恢复可继续处理的状态。

12.4 最终复验

部署完成后，按以下顺序验证：

代码示例 • POWERSHELL

```
pnpm demo:preflight
pnpm test:acceptance:deep
pnpm build
```

浏览器 E2E 和真实模型测试需要对应服务、浏览器、密钥和限额准备完成后再执行。最终结果记录到 `04-测试说明书与测试报告`，不能只保留终端截图。

资料真源

- `.env.example`
- `docker-compose.yml`

- package.json
- scripts/bootstrap-dev.mjs
- scripts/dev-all.mjs
- scripts/demo-serve.mjs
- scripts/demo-preflight.mjs
- prisma/schema.prisma
- server/core/ai/xunfei-adapter.ts
- docs/02-决策与开发过程记录/01-技术选型.md
- docs/02-决策与开发过程记录/03-桌面优先还是 web 优先? .md
- docs/02-决策与开发过程记录/09-根目录结构规则.md

— 文档结束 —